

GestEventos

Sistema de Gestão de Eventos e Ingressos

Documento de Visão e Diagramas do Projeto

Programação Orientada a Objetos — 2º Bimestre — 2026.1

Tecnologia em Análise e Desenvolvimento de Sistemas

IFRN — DIATINF

Autores

Pedro Henrique Graça de Assis Silva

Gustavo Samuel de Souza Silva

Natal/RN — julho de 2026

Conteúdo

1. Documento de Visão do Sistema
2. Diagrama de Casos de Uso
3. Diagramas de Classes — Modelo e Persistência
4. Diagramas de Classes — Serviço e Interface (View/Service)
5. Arquitetura em Camadas

Nota: este documento reúne a visão geral e os diagramas do projeto. O código-fonte completo (camadas model, persistence, service e view) e os testes automatizados encontram-se no repositório do projeto.

1. Documento de Visão do Sistema

1.1. Título do Projeto

GestEventos — Sistema de Gestão de Eventos e Ingressos

1.2. Objetivo do Sistema

O GestEventos tem por objetivo apoiar a organização, divulgação e comercialização de ingressos para eventos e conferências. O sistema permite que organizadores cadastrem eventos, definam lotes de ingressos (com preços e quantidades distintas) e criem cupons de desconto, enquanto participantes pesquisam eventos, inscrevem-se e geram seus ingressos, com aplicação automática de descontos e controle de vagas.

1.3. Descrição do Problema a Ser Resolvido

A venda de ingressos para eventos envolve regras que, quando controladas manualmente (planilhas, mensagens, anotações), geram erros frequentes:

- Controle de vagas: vender mais ingressos do que a capacidade de um lote (overbooking).
- Preços por lote: um mesmo evento costuma ter faixas de preço distintas (Lote 1, Lote 2, VIP), cada uma com quantidade própria e esgotamento independente.
- Descontos: aplicação de cupons promocionais de forma consistente e dentro da validade.
- Cálculo do valor final: o valor pago deve considerar o preço do lote menos o desconto.
- Rastreabilidade: relacionar cada ingresso ao participante, ao lote, ao evento e ao respectivo pagamento, permitindo validação (check-in) e eventual cancelamento.

O GestEventos centraliza essas informações e automatiza a regra de negócio central: ao efetivar uma inscrição, o sistema verifica a disponibilidade no lote, decrementa a quantidade disponível, gera o ingresso, aplica o cupom de desconto (se houver), calcula o valor final e registra o pagamento — tudo em uma única operação.

1.4. Perfis de Usuários Envolvidos

Perfil	Responsabilidades
Organizador	Cadastra categorias, locais, eventos, lotes de ingressos e cupons; valida ingressos (check-in); acompanha as inscrições dos seus eventos.
Participante	Cadastra-se, pesquisa eventos, realiza inscrições (compra de ingressos), aplica cupons, consulta e cancela seus ingressos.

O acesso é controlado por login e senha, e o menu de operações é exibido de acordo com o perfil do usuário autenticado.

1.5. Entidades do Sistema

Controle de usuários: Usuario (classe base) → Organizador e Participante.

Entidades de negócio:

- Categoria — classifica os eventos (ex.: Tecnologia, Música, Educação).
- Local — onde o evento acontece (endereço, cidade, capacidade).
- Evento — o evento/conferência propriamente dito.
- LoteIngresso — faixa de ingressos de um evento (ex.: Lote 1, Lote 2, VIP).
- CupomDesconto — cupom promocional aplicável a uma inscrição (opcional).
- Inscricao (Venda) — registro da compra/geração de um ingresso.
- Pagamento — pagamento associado a uma inscrição.

1.6. Lista de Operações do Aplicativo

1.6.1. Comuns (autenticação)

- Entrar no sistema (login com validação de perfil).
- Sair do sistema (logout).
- Cadastrar-se como participante.

1.6.2. Organizador

- CRUD de Categoria, Local, Evento, LotelIngresso e CupomDesconto.
- Associação: vincular Evento a Categoria/Local/Organizador; vincular LotelIngresso a Evento.
- Pesquisa: eventos por parte do título; eventos por categoria; inscrições de um evento.
- Regra de negócio: validar ingresso (check-in).

1.6.3. Participante

- Pesquisa: eventos por título ou categoria; lotes disponíveis de um evento.
- Regra de negócio — realizar inscrição: verifica disponibilidade, decrementa vaga, aplica cupom, calcula o valor final, gera o ingresso e registra o pagamento.
- Regra de negócio — cancelar inscrição: devolve a vaga ao lote e estorna o pagamento.
- Consultas: minhas inscrições/ingressos.

2. Diagrama de Casos de Uso

Atores Organizador e Participante, com CRUD de todas as entidades, entrar/sair, operações de associação, pesquisas e regras de negócio.

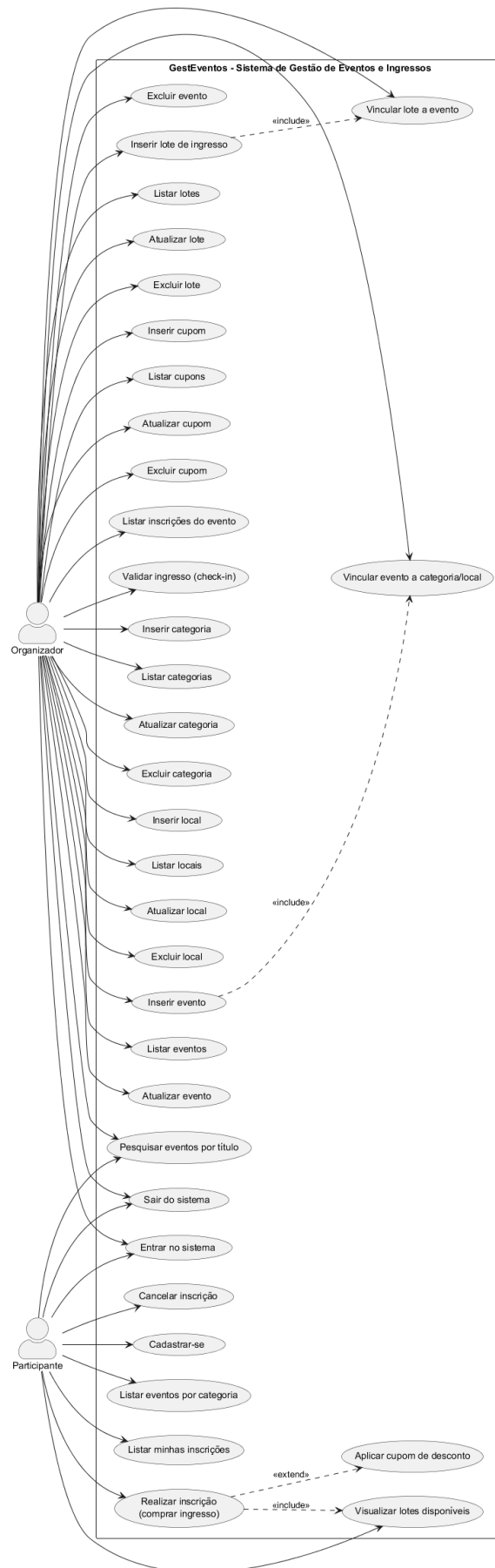


Figura 1 — Diagrama de Casos de Uso.

3. Diagramas de Classes — Modelo e Persistência

3.1. Diagrama de Classes do Modelo

Entidades de negócio, herança a partir de EntidadeBase e associações um-para-muitos (por id), com todos os membros das classes.

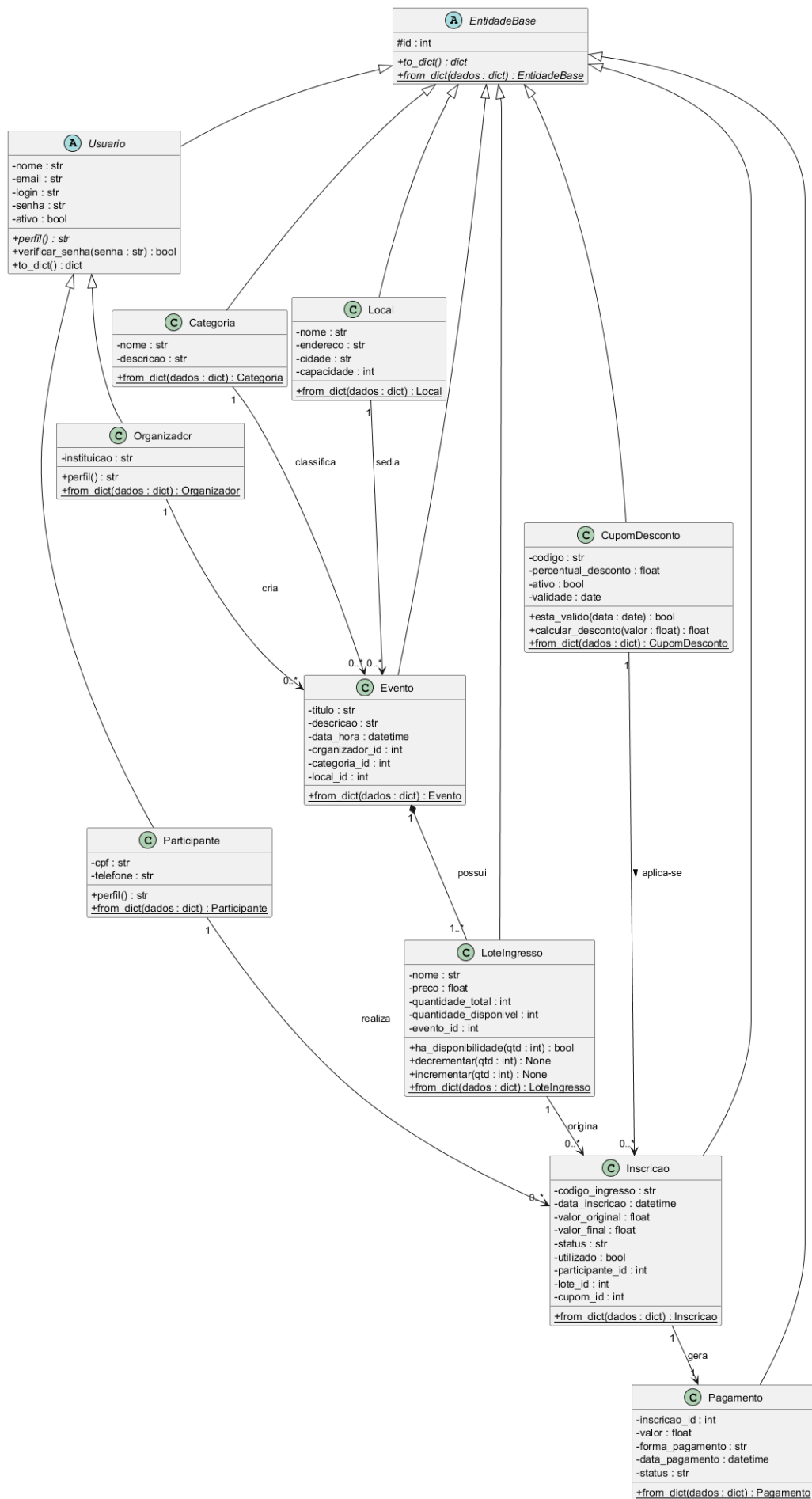


Figura 2 — Diagrama de Classes do Modelo.

3.2. Diagrama de Classes da Persistência

Repositório genérico RepositorioJson (CRUD) especializado por entidade, com as consultas de pesquisa e associação.

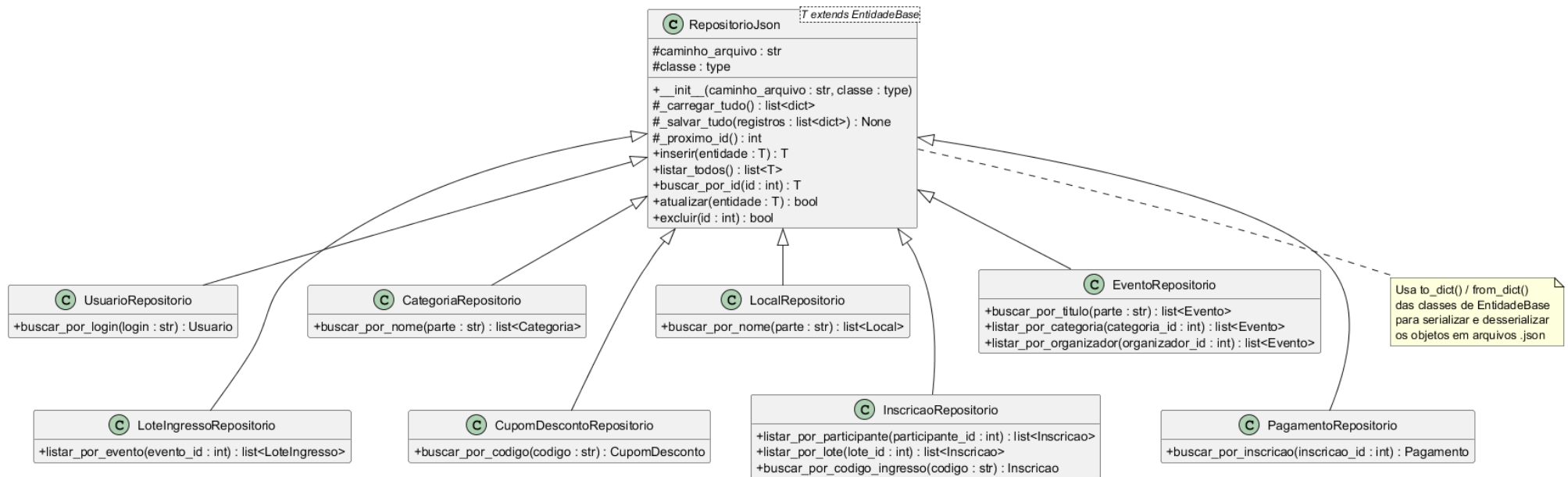


Figura 3 — Diagrama de Classes da Persistência.

4. Diagramas de Classes — Serviço e Interface

4.1. Diagrama de Classes de Operações (View/Service)

Camada de serviço: um serviço por entidade, mais o InscricaoService (regras de negócio de compra, cancelamento e check-in), reunidos pelo contêiner Servicos. Contém todos os métodos e parâmetros das operações.

Camada de Serviço (Operações e Regras de Negócio)

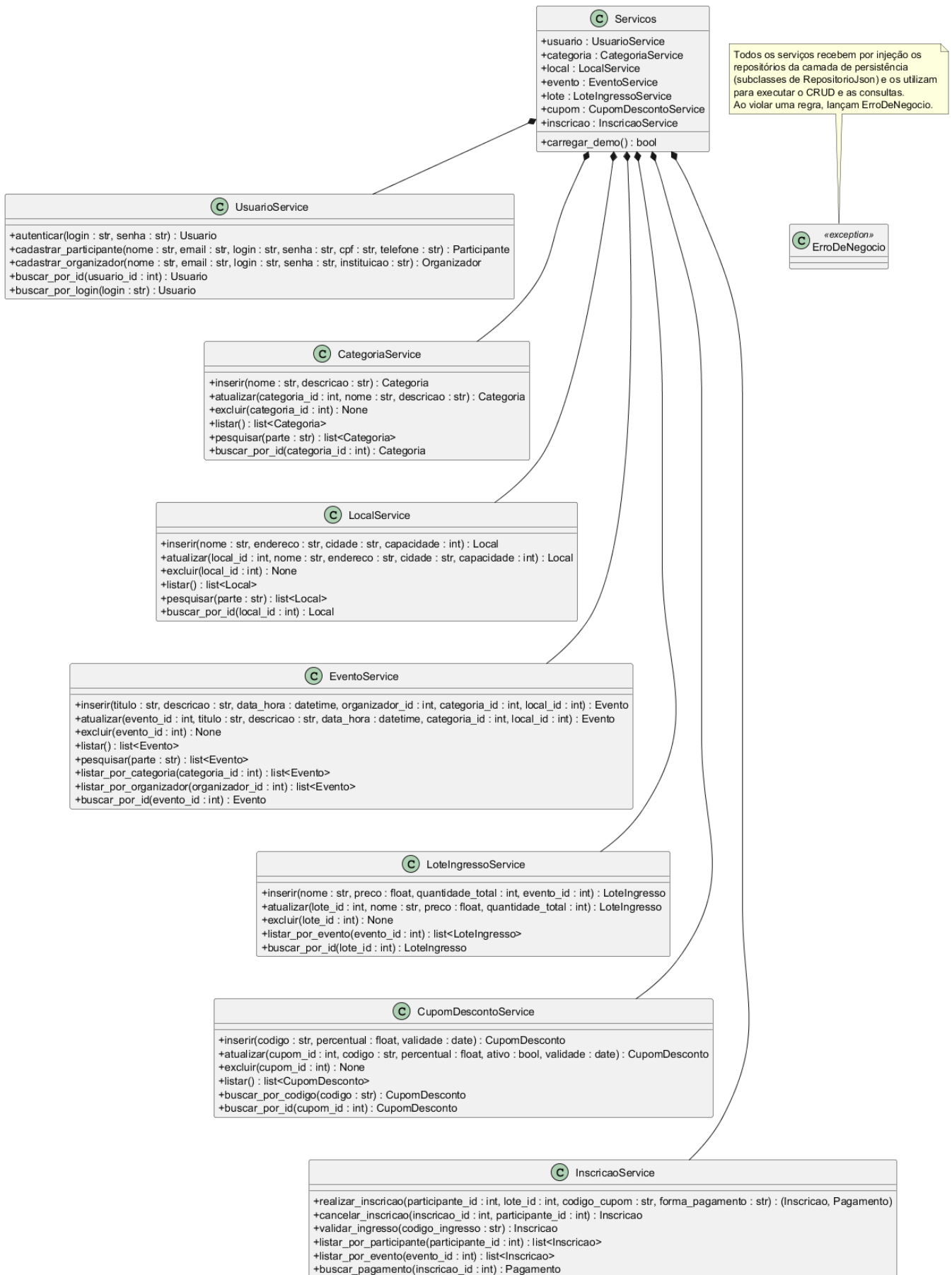


Figura 4 — Diagrama de Classes de Operações (View/Service).

4.2. Diagrama de Classes da Interface com o Usuário

Camada de interface: TelaBase (utilitários de entrada/saída) como superclasse das telas de login, dos menus por perfil e das telas de CRUD e de inscrição.

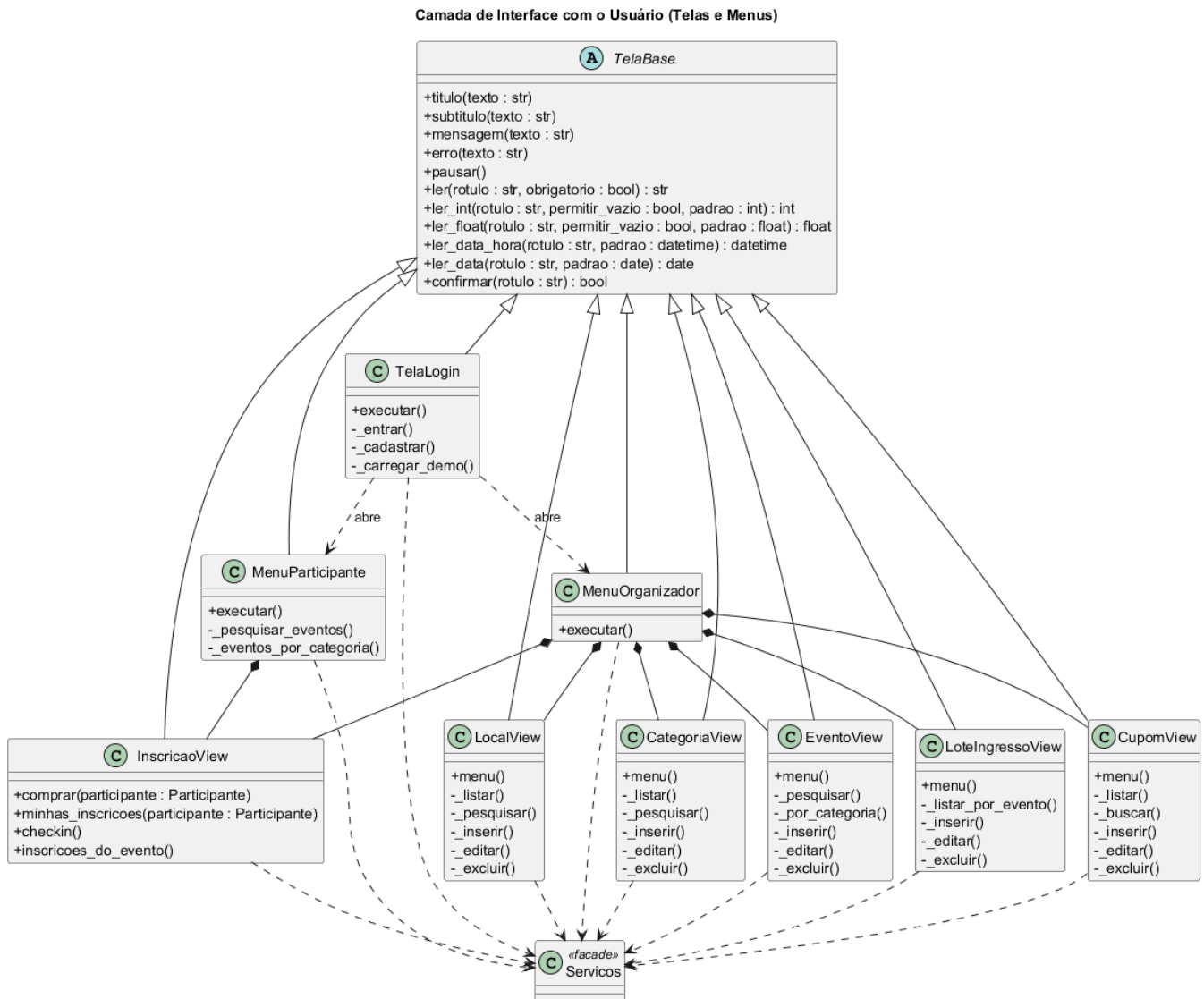


Figura 5 — Diagrama de Classes da Interface com o Usuário.

5. Arquitetura em Camadas

O sistema segue uma arquitetura em quatro camadas, com dependências sempre direcionadas de cima para baixo:

- **model** — entidades de negócio (Usuario e perfis, Evento, LotelIngresso, Inscricao, Pagamento, Categoria, Local, CupomDesconto), com encapsulamento, herança e polimorfismo.
- **persistence** — repositórios que estendem RepositorioJson e persistem os objetos em arquivos JSON via `to_dict()/from_dict()`.
- **service** — operações e regras de negócio; valida associações e lança `ErroDeNegocio` quando uma regra é violada. Não realiza entrada/saída com o usuário.
- **view** — interface de terminal: menus por perfil e telas de operação; delega toda a lógica à camada de serviço.

A execução do aplicativo é feita por `main.py`, que compõe as camadas (persistência → serviço → interface) e inicia a tela de login. O acesso por perfil garante que cada usuário veja apenas as operações permitidas.

Validação: o projeto acompanha testes automatizados das camadas de persistência e de serviço (CRUD, pesquisa, associação e regras de negócio de inscrição, cancelamento e check-in), todos aprovados.